Baseball Practice Session 1

Welcome to the first practice session of the semester! These sessions are meant to practice and reinforce the material taught in lecture. We will hold these sessions once a week, and each session will be 1 hour. Note: practice sessions will NOT be turned in.

In today's practice session, you'll practice:

1. Writing and evaluating some basic *expressions* in Python
2. Using pandas to manipulate baseball data through some guided exercises

Loading the Lahman Baseball Database

```
import YData_baseball

YData_baseball.download_data("Lahman_2024u/Batting.csv")
YData_baseball.download_data("Lahman_2024u/People.csv")
```

```
The file `Batting.csv` already exists.
If you would like to download a new copy of the file, please rename the
existing copy of the file.
The file `People.csv` already exists.
If you would like to download a new copy of the file, please rename the
existing copy of the file.
```

Loading the pandas and numpy Packages

```
import pandas as pd
import numpy as np
```

1. Quick Review of Python Data Types and Structures

1.1 Basic data types

Some of the main types of data in Python are:

- `int`: A whole number
- `float`: A decimal number
- `bool`: A logical value that can be True or False
- `strings`: A piece of text

We can also check what type of value is in a variable using the `type()` function.

1.2 Basic data structures

Some of the main data structures in Python are:

- `List`: my_list = [1, 2, 3]
- `Tuple`: my_tuple = (1, 2, 3)
- `Dictionary`:

  my_dictionary = {
  "key1": "value1",
  "key2": "value2",
  "key3": "value3"
  }

1.3 Pandas DataFrames

The most applicable data structure for data science is the DataFrame, which is basically a spreadsheet in Excel. The Pandas package allows us to use DataFrames. Here is the players DataFrame. We can look at the first five rows by adding `.head(5)` to the end.

```
batting = pd.read_csv("Batting.csv")
batting.head(5)
```

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H	...	SB	CS	BB	SO	IB
0	aardsda01	2004	1	SFN	NL	11	NaN	0	0	0	...	0.0	0.0	0	0.0	0.0
1	aardsda01	2006	1	CHN	NL	45	NaN	2	0	0	...	0.0	0.0	0	0.0	0.0
2	aardsda01	2007	1	CHA	AL	25	NaN	0	0	0	...	0.0	0.0	0	0.0	0.0

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H	...	SB	CS	BB	SO	IB
3	aardsda01	2008	1	BOS	AL	47	NaN	1	0	0	...	0.0	0.0	0	1.0	0.0
4	aardsda01	2009	1	SEA	AL	73	NaN	0	0	0	...	0.0	0.0	0	0.0	0.0

1.4 Numpy Arrays

Numpy arrays are similar to lists, but all elements in a numpy array must be of the same type (e.g., all integers, all strings).

2. Data Structure Practice

2.1 Creating a dictionary

Create a dictionary that maps each of the following abbreviations to their full name: 'G', 'G_batting', 'AB', 'R', 'H', '2B', '3B', 'HR', 'RBI', 'SB', 'CS', 'BB','SO','IBB', 'HBP','SH','SF', and 'GIDP'.

```
# Answer
player_dict = {
    'G': "Games Played",
    'G_batting': "Games Played (Batting)",
    'AB': "At-Bats",
    'R': "Runs Scored",
    'H': "Hits",
    '2B': "Doubles",
    '3B': "Triples",
    'HR': "Home Runs",
    'RBI': "Runs Batted In",
    'SB': "Stolen Bases",
    'CS': "Caught Stealing",
    'BB': "Base on Balls",
    'SO': "Strikeouts",
    'IBB': "Intentional Base on Balls",
    'HBP': "Hit By Pitch",
    'SH': "Sacrifice Hits",
    'SF': "Sacrifice Flies",
    'GIDP': "Grounded Into Double Play"
}
```

2.2 Creating a Numpy Array

Convert the following lists to arrays. Then calculate the walks per plate appearance. Round your answer to two decimal places.

```
import numpy as np

# Lists showing Mike Trout's seasons, walks, and plate appearances
seasons = [2014, 2015, 2016, 2017, 2018]
walks = [83, 92, 116, 94, 122]
plate_appearances = [705, 682, 681, 507, 608]

# Answer
# convert the lists to numpy arrays
walks_array = np.array(walks)
plate_appearances_array = np.array(plate_appearances)
walks_per_PA = (walks_array / plate_appearances_array)
np.round(walks_per_PA, 2)
```

```
array([0.12, 0.13, 0.17, 0.19, 0.2 ])
```

2.3 Selecting Columns from a DataFrame

Consider the players DataFrame. Select the columns `playerID`, `nameGiven`, `birthYear`, and `birthState`.

```
players = pd.read_csv("People.csv")

# Answer
players[['playerID', 'nameGiven', 'birthYear', 'birthState']]
```

	playerID	nameGiven	birthYear	birthState
0	aardsda01	David Allan	1981.0	CO
1	aaronha01	Henry Louis	1934.0	AL
2	aaronto01	Tommie Lee	1939.0	AL
3	aasedo01	Donald William	1954.0	CA
4	abadan01	Fausto Andres	1972.0	FL
...	...	...	...	...
24018	zupofr01	Frank Joseph	1939.0	CA
24019	zuvelpa01	Paul	1958.0	CA

	playerID	nameGiven	birthYear	birthState
24020	zuverge01	George	1924.0	MI
24021	zwilldu01	Edward Harrison	1888.0	MO
24022	zychto01	Anthony Aaron	1990.0	IL

2.4 Subsetting Rows in a DataFrame

Looking again at the players DataFrame, write code to select the players that were born in Denver.

```
# Answer
players[players['birthCity'] == 'Denver'].head(5)
```

	ID	playerID	birthYear	birthMonth	birthDay	birthCity	birthCountry	birthState	deatl
0	1	aardsda01	1981.0	12.0	27.0	Denver	USA	CO	NaN
169	153	akerfda01	1962.0	6.0	12.0	Denver	USA	CO	2012
418	21308	anderbu02	1904.0	11.0	4.0	Denver	USA	CO	1943
1732	1506	blachty01	1990.0	10.0	20.0	Denver	USA	CO	NaN
1830	1584	blatest01	1944.0	3.0	20.0	Denver	USA	CO	NaN

2.5 Using Methods on Columns in a DataFrame

The pandas Series and DataFrames also have methods that can operate on the values in a column. For example, we can sum the values in a column using the my_df.sum() method. We can also use numpy functions on columns of a DataFrame if we extract the data in the columns as pandas Series. For we can also sum the values in a column of a pandas DataFrame using `np.sum(my_df["col_name"])`.

Looking back at the batting data, what is the total number of stolen bases from 1871 through the 2024 season? Please calculate this value using both the pandas .sum() method and the numpy np.sum() function

Answer There have been 339554 stolen bases from 1871 through 2024.

```
import numpy as np

# Answer
# Using the pandas .sum() method
print(batting['SB'].sum())
```

```
# Using numpy np.sum()
np.sum(batting["SB"])
```

```
339554.0
```

```
np.float64(339554.0)
```

2.6 Boolean indexing and the .query() methods

As we discussed extensively in YData, you can use Boolean indexing and/or the .query() method to extract a DataFrame that only contains rows that meet particular conditions. For example, if we have a DataFrame called my_df that has a column called vals that has quantitative data, you could extract a DataFrame that only has rows where vals are greater than 10 using either:

- `my_df[my_df["vals"] > 10]`
- `my_df.query("vals > 10")`

2.7 Aggregate statistics by group

We can use the pandas groupby() and .agg() method to calculate aggregate statistics separately for different groups. For example, we can had a DataFrame called my_df that had a categorical column called "group_col" and a quantitative column called "quant_col", we could calculate the sum of the values in the "quant_col" separately for each unique value in the "group_col" using `my_df.groupby("group_col").agg(sum_vals = ("quant_col", "sum"))`

Using the batting data, please calculate the average ("mean") number of strike outs for each season from 1871 to 2024. Also, sort the values such that the season that has the most strike outs is at the top of this aggregated DataFrame.

```
# Answer
batting.groupby("yearID").agg(mean_SO = ("SO",
 ↪  "mean")).sort_values("mean_SO", ascending = False)
```

yearID	mean_SO
2019	27.293180
2018	26.844951
2017	26.843373

yearID	mean_SO
1968	26.773427
2016	26.285907
...	...
1875	3.110599
1874	2.902439
1873	2.224000
1872	1.687898
1871	1.521739

3. Player Case Study: Mike Trout

Mike Trout is an American MLB player with an elite regular-season resume. He won the American League MVP award three times (2014, 2016, 2019), and is an 11-time All-Star. Our analysis will focus on these three seasons, and see how they compare to the league average.

3.1 Pulling Season Data

Subset the batting data to look at the 2014, 2016, and 2019 season. Save this to a DataFrame called `batting_14_16_19`.

```
# Answer
batting_14_16_19 = batting[batting["yearID"].isin([2014, 2016, 2019])]

batting_14_16_19.head(5)
```

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H	...	SB	CS	BB	SO
59	abadfe01	2014	1	OAK	AL	69	NaN	0	0	0	...	0.0	0.0	0	0.0
61	abadfe01	2016	1	MIN	AL	39	NaN	1	0	0	...	0.0	0.0	0	1.0
62	abadfe01	2016	2	BOS	AL	18	NaN	0	0	0	...	0.0	0.0	0	0.0
64	abadfe01	2019	1	SFN	NL	21	NaN	0	0	0	...	0.0	0.0	0	0.0
253	abreubo01	2014	1	NYN	NL	78	NaN	133	12	33	...	1.0	0.0	20	21.0

3.2 Pulling Trout's MVP Data

Using the `batting_14_16_19` data, filter the data to look at Trout. Note his `playerID` is `troutmi01`.

```
# Answer
trout_mvp = batting_14_16_19[batting_14_16_19['playerID'] == 'troutmi01']
trout_mvp
```

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H	...	SB	CS	BB
115313	troutmi01	2014	1	LAA	AL	157	NaN	602	115	173	...	16.0	2.0	83
115315	troutmi01	2016	1	LAA	AL	159	NaN	549	123	173	...	30.0	7.0	11
115318	troutmi01	2019	1	LAA	AL	134	NaN	470	110	137	...	11.0	2.0	11

3.3 Calculating League-Wide Summary Statistics

Using the Numpy `.mean()` method and the `batting_14_16_19` data, calculate the average runs, hits, home runs, stolen bases, and RBIs for the league.

```
# Answer
# League
print("R mean:", np.mean(batting_14_16_19["R"]))
print("H mean:", np.mean(batting_14_16_19["H"]))
print("HR mean:", np.mean(batting_14_16_19["HR"]))
print("SB mean:", np.mean(batting_14_16_19["SB"]))
print("RBI mean:", np.mean(batting_14_16_19["RBI"]))
```

```
R mean: 14.4800534878538
H mean: 28.061065299754848
HR mean: 3.693336304880767
SB mean: 1.6895475819032761
RBI mean: 13.809003788722977
```

3.4 Calculating Trout's Summary Statistics

Repeat the above analysis using the Trout data.

```
# Answer
# Trout
print("R mean:", np.mean(trout_mvp["R"]))
print("H mean:", np.mean(trout_mvp["H"]))
print("HR mean:", np.mean(trout_mvp["HR"]))
print("SB mean:", np.mean(trout_mvp["SB"]))
print("RBI mean:", np.mean(trout_mvp["RBI"]))
```

```
R mean: 116.0
H mean: 161.0
HR mean: 36.666666666666664
SB mean: 19.0
RBI mean: 105.0
```

3.5 Comparing Trout to the League

Compare Trout's performance to that of the league. What are pros to this analysis? What are some cons?

Answer

Pros: - We can quickly compare Trout's performance to the average MLB player - And we can see that his numbers are far greater

Cons: - Taking an average doesn't take into account the shapes of the distributions (skewed vs symmetric) - Lumping the entire league together doesn't account for different positions or injured players - Grouping seasons together can be misleading due to season-level differences: Trout may have been far better than other players in one season, but only marginally better in another

```
%%capture

# You can run this code to covert this Jupyter notebook into a pdf
!quarto render practice_1_answers.ipynb --cache-refresh --to pdf
```